



ALBUKHARY INTERNATIONAL UNIVERSITY
DU014 (K)

ALBUKHARY INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATICS
CCS2243 – CRYPTOGRAPHY ESSENTIAL
**VIGENÈRE CIPHER IMPLEMENTATION AND CRYPTANALYSIS OF UNKNOWN-
KEY CIPHERTEXT**

Submitted By

BENAT SIRAJ AHMED AIU24102456

Class: BCS2B

Instructor:

PROF. DR. KHALID HUSSAIN

Submission Date:

18 September 2026

Table of Contents

Table of Contents

Abstract	4
Chapter 1: Introduction	5
1.1 Background	5
1.2 Problem statement	5
1.3 Aim and objectives	5
1.4 Scope	5
1.5 Security goals and mode definition	6
Chapter 2: Literature Review and Cryptographic Background	7
2.1 Classical polyalphabetic substitution	7
2.2 Kasiski Examination	7
2.3 Index of Coincidence	7
2.4 Chi-squared frequency analysis	7
2.5 Security limitations relative to modern cryptography	7
Chapter 3: System Design	9
3.1 Functional and security requirements	9
3.2 UML Use Case Diagram	9
3.3 System Flowchart	9
3.4 System Architecture and Data Flow	10
Chapter 4: Implementation	12
4.1 Development environment	12
4.2 Core cipher functions	12
4.3 Cryptanalysis functions	12
4.4 Console-level interface	12
4.5 Controlled test corpus	13
Chapter 5: Testing and Evaluation	14
5.1 Round-trip correctness	14
5.2 Plaintext and ciphertext letter distribution	14
5.3 Index of Coincidence across candidate key lengths	14
5.4 Cryptanalysis accuracy versus ciphertext length	15
5.5 Performance benchmark	16
5.6 Edge cases and negative tests	17

5.7 Security evaluation by mode.....	17
Chapter 6: Results and Discussion.....	18
Chapter 7: Limitations and Responsible Use.....	19
Chapter 8: Conclusion.....	20
References.....	21
Appendix.....	22
Appendix A: Source Code	22
Appendix B: Demonstration Checklist	22
Appendix C: Key Implementation Constants	22
Appendix D: Generative AI Acknowledgement.....	22
Appendix E: Console and CrypTool 2 Screenshots.....	22

List of Figures

- Figure 1. UML use case diagram for the Vigenère Cipher System.
- Figure 2. System flowchart showing encrypt, decrypt, and break paths.
- Figure 3. System architecture and data flow between input, cipher engine, cryptanalysis engine, and testing.
- Figure 4. Data flow diagram (Level 0).
- Figure 5. Letter frequency comparison: plaintext versus ciphertext.
- Figure 6. Index of Coincidence by candidate key length.
- Figure 7. Cryptanalysis accuracy versus ciphertext length.
- Figure 8. Performance benchmark by operation and input size.
- Figure 9. Console -- Mode 1 encrypt, textbook vector.
- Figure 10. Console -- Mode 2 decrypt, confirming the round trip.
- Figure 11. Console -- Mode 3 break ciphertext, unknown key recovered.
- Figure 12. CrypTool 2 -- Kasiski/Index of Coincidence analysis.
- Figure 13. GitHub Repository of the Vigenère Cipher Implementation and link

List of Tables

- Table 1. Security goals and mode definition.
- Table 2. Functional and security requirements.
- Table 3. Round-trip correctness results.
- Table 4. Cryptanalysis accuracy versus ciphertext length (raw results).
- Table 5. Performance benchmark (raw results).
- Table 6. Edge case and negative test results.
- Table 7. Security evaluation by mode.
- Table 8. Key implementation constants.

VIGENÈRE CIPHER IMPLEMENTATION AND CRYPTANALYSIS OF UNKNOWN-KEY CIPHERTEXT

Abstract

The Vigenère Cipher Implementation and Cryptanalysis project was designed to demonstrate to the students the application of classic encryption techniques as well as, importantly, the statistical approach to cryptanalysis of the Vigenère cipher when the key is unknown in CCS2243 Cryptography Essential individual assessment. The system was implemented in Python as a standalone script with 2 modes of work.

Mode 1: The classic Vigenère cipher encryption and decryption where the key is known and implemented as follows: $C_i = (P_i + K_i) \bmod 26$.

Mode 2: Cryptanalysis – the system works only with the ciphertext and without the key; it estimates the length of the key based on the Index of Coincidence, finds the frequency of each letter of the key from the table of frequencies of English letters and gets the key, and then cracks the ciphertext to get the plaintext.

The system was tested against a known vector of the textbook cipher, mixed case and punctuated text, and a test corpus of 673 characters. All the tests were verified for round-trip correctness by comparing strings directly. Other tests include the distribution of the letter frequency, the Index of Coincidence for different lengths of the keys, the accuracy of cryptanalysis as a function of the length of the ciphertext, the encryption and decryption as well as the attack performance as a function of the length of the input from 500 to 50,000 characters, and handling of the edge cases like invalid keys, very short input lengths, and wrong decryption keys.

The attack of the cryptanalysis proved successful in getting the exact 6-letter key (CIPHER) and the full plaintext as soon as the length of the CIPHER exceeded the threshold of about 240 letters; for a very short CIPHER (11 letters) the attack was unsuccessful, as mentioned in the report, but this is not treated as a certain limit that is not being enforced, just a limit of the attack. All in all, this project demonstrates both aspects of CLO3: proper use of the cryptographic technique, and the knowledge acquired – from the code and physical measurements – of the conditions in which its guarantee of confidentiality fails.

Chapter 1: Introduction

1.1 Background

Classical ciphers used to be thought of as being secure since they cannot be cracked through the simple letter frequency analysis attacks, used against monoalphabetic substitution ciphers. The statistical features that would be exploited by an attacker are masked by mapping one letter of the plaintext to many different letters in the ciphertext depending on its position in the original text. Centuries ago it was called “le chiffre indéchiffrable” (the indecipherable cipher).

This project was built in order to teach two historical aspects of the cipher implementation: Implementation of the cipher in a correct way, and its cryptanalysis and cracking using statistical methods, implemented in working code and with measurable results, not just theoretically postulated.

1.2 Problem statement

The security specialists at an ecommerce company find old internal logs encrypted with a simple cipher that works through the shifts. The key has been lost and now the logs have to be analyzed and decrypted without any knowledge of the original key. Without also implementing cryptanalysis, basic encryption cannot answer the question presented in this problem statement: is the plaintext actually recoverable, and if it is, how much ciphertext is needed for statistically viable recovery? This project will deal with it directly by developing both the cipher and the attack on it and then testing the effectiveness of the attack.

1.3 Aim and objectives

The main goal of this Vigenère Cipher project is to implement encryption and decryption and explore how secure the cipher is.

There for it built develop and then test a running Vigenère cipher and cryptanalysis modules that will be able to break an unknown key based on ciphertext.

To implement Vigenère encryption and decryption correctly for any text, mixed case and punctuation included.

2. To estimate the length of the key based on the Index of Coincidence.

To obtain each letter of an unknown key from standard English letter frequencies with the chi-squared frequency analysis.

4. To verify implementation correctness by testing with a known vector of the textbook cipher and round-trip comparison.

5. To investigate the effect of the length of the ciphertext on the accuracy of the cryptanalysis.

6. To measure performance on attacks, encryption and decryption for various input sizes.

To look into edge cases like invalid keys, too short ciphertext, incorrect decryption keys.

8. To evaluate the actual practical weaknesses of the cipher in terms of security from the data collected throughout the project.

1.4 Scope

The program accepts only strings containing the English alphabet (A-Z). It will receive as a parameter either plaintext or ciphertext as string and keyword for encrypt/decrypt and ciphertext only for cryptanalysis mode. It is a standalone script, written in Python without any dependence on the external cryptographic libraries because all algorithm and attack implementation are done by myself. The aim of the program is not in educating students how to use general text-

encryption programs. It is just a demonstration of classical cipher and its cryptanalysis. Also modern encryption algorithms such as AES or RSA are out of the scope of this project.

1.5 Security goals and mode definition

Mode	Main technique	Primary security goal	Important limitation
1	Vigenère encryption/decryption with a known key	Confidentiality (against a naive reader)	No integrity or authentication guarantee
2	Cryptanalysis: Index of Coincidence + chi-squared frequency analysis	Demonstrates failure of confidentiality without the key	Requires sufficient ciphertext length (~240+ letters) to be reliable

Table 1. Security goals and mode definition.

Chapter 2: Literature Review and Cryptographic Background

2.1 Classical polyalphabetic substitution

Vigenère Cipher was invented by Giovan Battista Bellaso in 1553 but mistakenly named after French diplomat Blaise de Vigenère (Kahn, 1996). It uses the following formula for encryption: $C_i = (P_i + K_i) \bmod 26$, and decryption $(C_i - K_i) \bmod 26$, where key is repeated cyclically through the whole message. As it repeats, the same plaintext letter can have different values in the ciphertext letters according to its position. That is why it cannot be deciphered using standard frequency analysis (Singh, 1999).

Problem statement of this project is based on the lab exercise in Week 2. Lab 9 (Vigenère Cipher) introduces us to the mechanics of encryption and decryption used in section 4.2. Also the Task #1 of Lab 3 (Cryptanalysis and Brute-Force Attack) poses the general problem of obtaining plaintext from classical cipher without the knowledge of the key. The problem is extended here from manual/brute-force recovery of single-shift cipher to the statistical attack on polyalphabetic key of unknown length using Index of Coincidence (section 2.3) and chi-squared frequency analysis (section 2.4), as done in section 4.3.

2.2 Kasiski Examination

Friedrich Kasiski developed the first general technique for attacking the cipher in 1863 (Kasiski, 1863, as cited in Kahn, 1996). Repeated sequences of letters and distance between them usually have some common divisors with the key length. That means that identical plaintext sequences encrypted at positions which are divisible by the key length will produce identical ciphertext sequences. This helps attacker to find candidate key lengths without knowing the key itself.

2.3 Index of Coincidence

The Index of Coincidence was proposed as a better statistical test compared to the Kasiski test by William F. Friedman (Friedman, 1922). It measures the probability that two randomly selected letters in the text are equal. English language texts have an Index of Coincidence of about 0.067, since the letters are unevenly distributed (the most frequent letters are E, T, A, O, I, N), whereas random and misaligned texts have an Index of Coincidence of about 0.038. Breaking a ciphertext into N interleaved parts with respect to each N as a potential key length and finding out which N results in parts with similar to English average Index of Coincidence allows one to reliably find the correct key length. This project uses exactly this algorithm in the `estimate_key_length()` function described in chapter 4.

2.4 Chi-squared frequency analysis

Once we have estimated the key length, every interleaved part of the text becomes a simple Caesar shifted part of English plaintext. In order to decipher it, we try all 26 possible shifts and compare the letter distribution produced by each shift against standard English letter frequencies using a chi-squared statistic, thus the shift producing the lowest score, or the most English-like letter distribution, is the correct one (Stallings, 2020). Finally, we combine all deciphered shifts together in order to restore the whole key.

2.5 Security limitations relative to modern cryptography

The Vigenère cipher provides confidentiality only to the attacker ignorant of cryptanalysis, but does not offer any integrity, authentication or non-repudiation, and, even more importantly, the confidentiality of the message can be broken, once there is enough ciphertext. All these problems

are solved by modern cryptographic algorithms, such as AES (providing confidentiality with a much larger key space), HMAC (providing integrity and authentication) and digital signatures (providing non-repudiation) — the concepts inspired by the cryptanalysis presented in this project.

Chapter 3: System Design

3.1 Functional and security requirements

Req.	Description
R1	The user can choose to encrypt, decrypt, or break ciphertext with an unknown key.
R2	The system encrypts and decrypts arbitrary text, including mixed case and punctuation, without corrupting the key alignment.
R3	Encryption and decryption use the standard formula $C_i = (P_i + K_i) \bmod 26$ and its inverse.
R4	The cryptanalysis mode estimates key length using the Index of Coincidence without being given the key.
R5	The cryptanalysis mode recovers each key letter using chi-squared frequency analysis.
R6	Invalid keys (no alphabetic characters) are rejected with a clear error.
R7	The system can compare recovered plaintext against the original to confirm exact recovery.
R8	The project records correctness, timing, letter-distribution, and accuracy-versus-length evidence.

Table 2. Functional and security requirements.

3.2 UML Use Case Diagram

Distinguish between user action and cryptanalysis action using use case diagram. Input is selected by user, then he/she performs the encryption, decryption, and/or cryptanalysis of cipher text. In the process of cryptanalysis, key length estimation and extraction of the key is done internally..

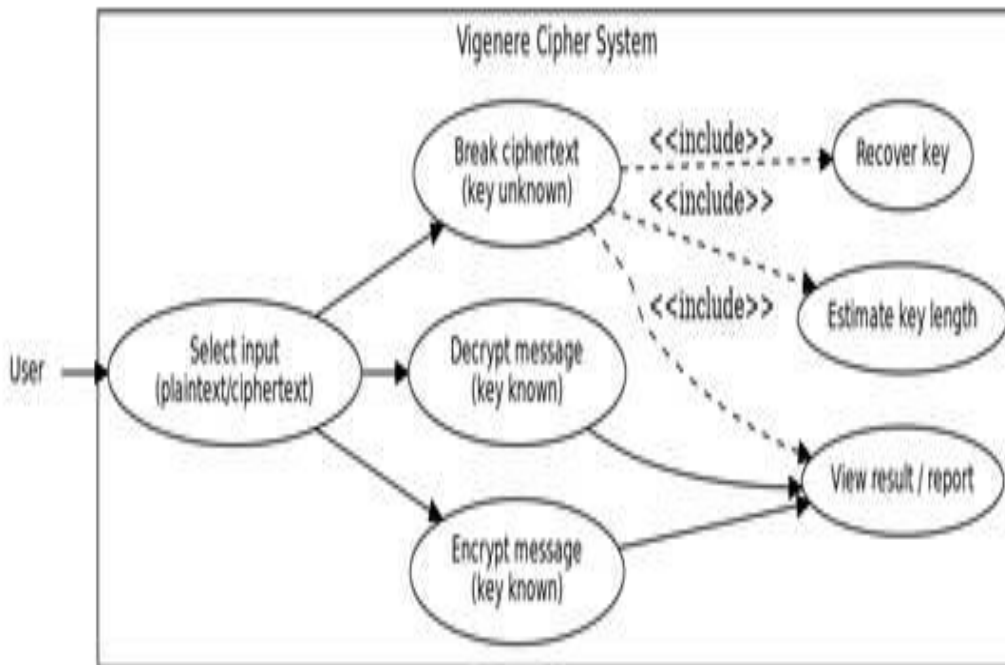


Figure 1. UML use case diagram for the Vigenère Cipher System.

3.3 System Flowchart

This flowchart follows the entire logic of the program through all the three processes; encryption, decryption, and cryptanalysis.

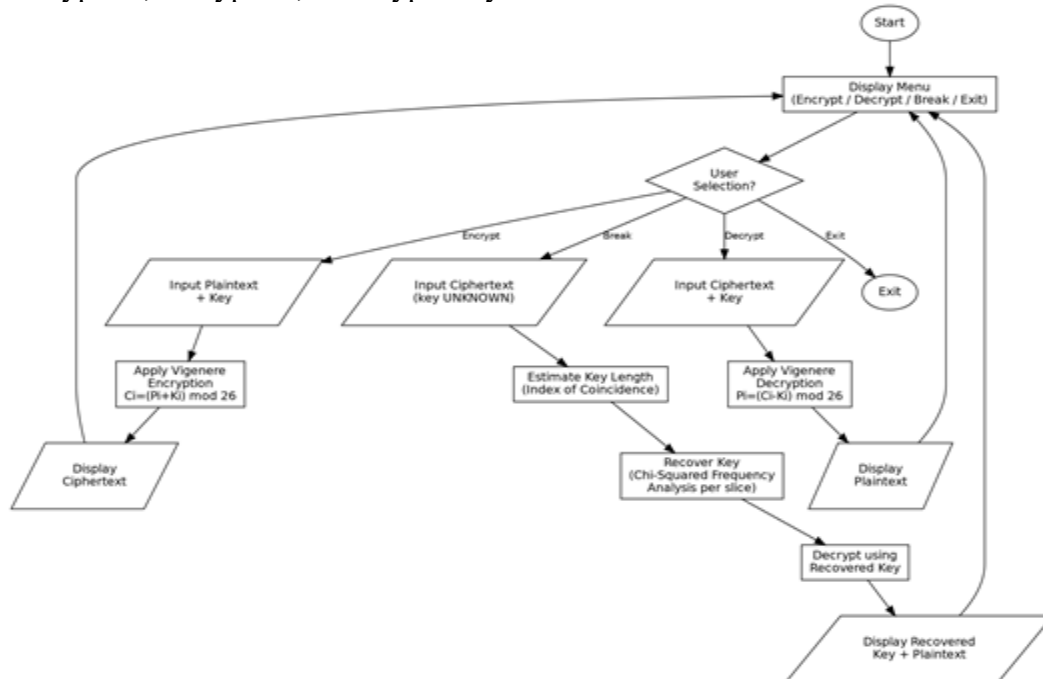


Figure 2. System flowchart showing encrypt, decrypt, and break paths.

3.4 System Architecture and Data Flow

The separation of design makes the cipher engine, the cryptanalysis engine, and the testing/measurement logic independent of one another to enable independent testing of the cryptographic algorithm.

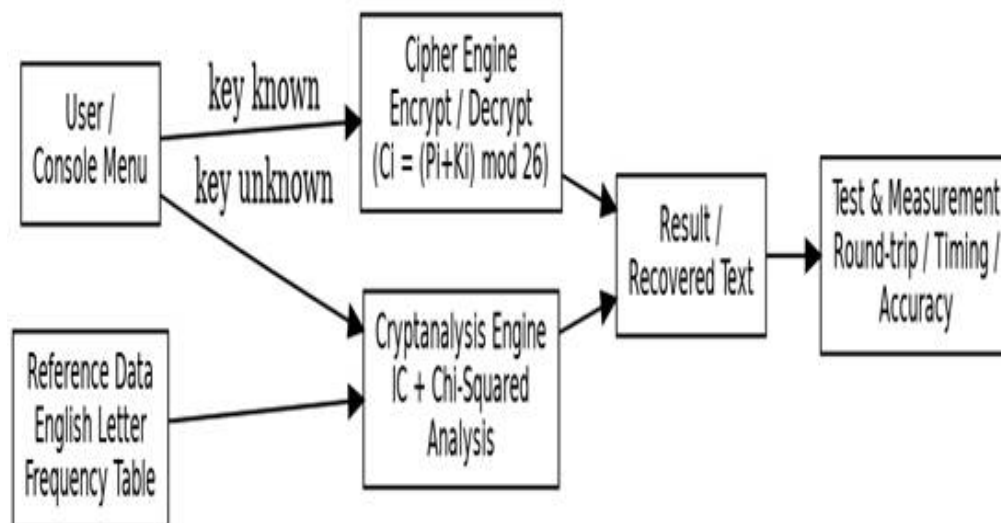


Figure 3. System architecture and data flow between input, cipher engine, cryptanalysis engine, and testing.

The below-given data flow diagram represents the same separation from a data point of view, where the static English letter frequency table is used by the cryptanalysis branch alone.

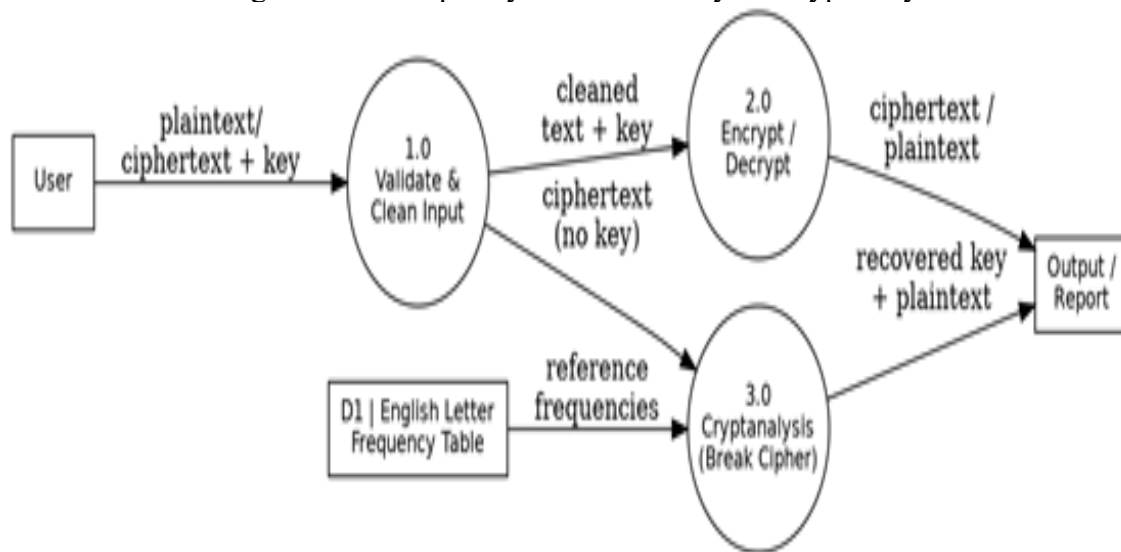


Figure 4. Data flow diagram (Level 0).

Chapter 4: Implementation

4.1 Development environment

The implementation is a single script written in Python 3 and using only the standard library (string manipulation and collections.Counter). No external cryptographic libraries are used, as the Vigenère cipher and its cryptanalysis are performed from scratch in order to demonstrate knowledge of the mathematics behind the method. Matplotlib is utilized separately in order to produce the evidence charts shown in Chapter 5.

4.2 Core cipher functions

The `clean_key()` function removes non-alphabetic characters and normalizes the case of the key. The `vigenere_encrypt()` and `vigenere_decrypt()` functions use the formulas above for each letter, while non-alphabetic characters (spaces, punctuation marks, etc.) pass through unencrypted without consuming a key letter:

Encryption: $C_i = (P_i + K_i) \bmod 26$

Decryption: $P_i = (C_i - K_i) \bmod 26$

```
def vigenere_encrypt(plaintext, key):
    key = clean_key(key)
    result, key_index, key_len = [], 0, len(key)
    for char in plaintext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            p = ord(char.upper()) - ord('A')
            k = ord(key[key_index % key_len]) - ord('A')
            result.append(chr((p + k) % 26 + base))
            key_index += 1
        else:
            result.append(char)
    return "".join(result)
```

4.3 Cryptanalysis functions

The `index_of_coincidence()` calculates the index of coincidence of an alphabetic-only string. The `estimate_key_length()` function splits the ciphertext into N interleaved slices for every candidate key length and returns the smallest N whose average IC exceeds the English-like threshold (0.058), as multiple of the key length scores well too:

```
def index_of_coincidence(text):
    freqs = Counter(text)
    n = len(text)
    numerator = sum(f * (f - 1) for f in freqs.values())
    return numerator / (n * (n - 1))
```

The `recover_key_letter()` function attempts every possible 26 letter shift for a particular slice and returns the letter shift producing the closest letter distribution to the standard English one using the chi-squared distance. The `recover_key()` function performs this task for all slices and reconstructs the whole key. The `break_vigenere()` function combines the two into a single attack pipeline.

4.4 Console-level interface

A simple menu presents the user with four options: encryption of a message, decryption of a message with a known key, breaking the ciphertext with an unknown key, and exiting. The invalid keys are handled by repeating the prompt until a valid one is entered with `get_valid_key()` instead of crashing.

4.5 Controlled test corpus

A public corpus does not apply in this project as there is no training of any model nor processing of records from external sources. A specially-written controlled English corpus has been developed as the expected letter frequencies and plaintext is known in advance in order to objectively measure cryptanalysis results.

Corpus	Letters	Purpose
Business-report style passage	579	Round-trip correctness, IC measurement, full attack demonstration
Truncated versions of the same passage	30 to 579	Accuracy-versus-length experiment (Section 5.4)
Randomly generated pseudo-English text	500 to 50,000	Performance benchmarking (Section 5.5)

Chapter 5: Testing and Evaluation

5.1 Round-trip correctness

The implementation was checked against a known textbook cipher vector, a message containing both uppercase/lowercase letters and punctuation marks, and the full 579-letter corpus.

Test	Expected	Result
ATTACKATDAWN / LEMON (Kahn, 1996)	LXFOPVEFRNHR	Pass — exact match
Mixed case + punctuation round-trip	Original text exactly	Pass — exact match
Key longer than message (HI / LONGKEYWORD)	HI	Pass — exact match
Full 579-letter corpus, key CIPHER	Original text exactly	Pass — exact match

Table 3. Round-trip correctness results.

5.2 Plaintext and ciphertext letter distribution

The 579-letter corpus was encoded using the key CIPHER. The distribution of plaintext letters shows clear peaks (E, T, and other English language letters); the distribution of ciphertext letters is clearly more uniform, though not perfectly so since the length of the key (6) is rather short compared to the alphabet.

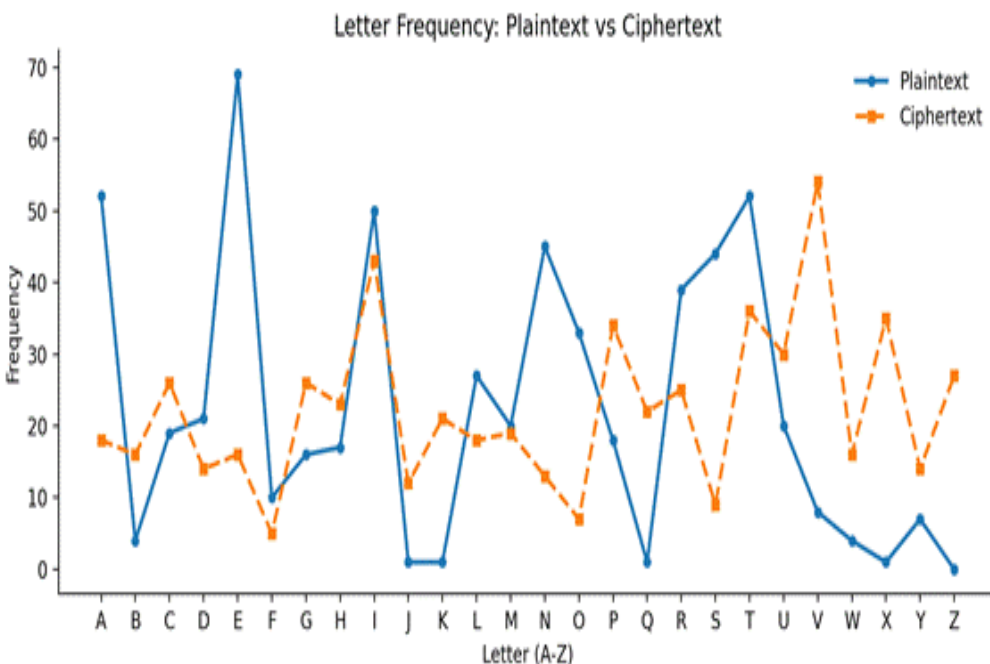


Figure 5. Letter frequency comparison: plaintext versus ciphertext.

The measured index of coincidence confirms this visual impression: the plaintext IC is equal to 0.0661 (which is close to the theoretical English language IC of 0.067), while the ciphertext IC is equal to 0.0465 (which is between English language IC and random IC, corresponding to the partial flattening caused by a 6-letter key).

5.3 Index of Coincidence across candidate key lengths

Running the `estimate_key_length()` function against the same ciphertext for various candidate lengths between 1 and 12 results in the following output, correctly estimating length 6 as the

smallest candidate meeting the English-like threshold:

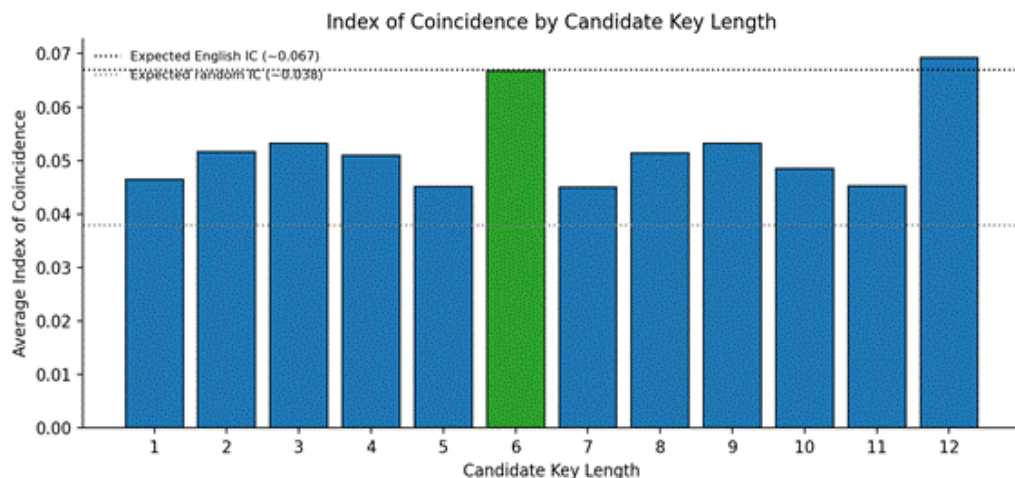


Figure 6. Index of Coincidence by candidate key length.

Length 12 (being an exact multiple of the real length) passes the threshold as well; this is expected: every subcipher resulting from splitting ciphertext into slices of a multiple length remains a plain Caesar shift. Choosing the smallest such length ensures that an excessively long key is not returned.

5.4 Cryptanalysis accuracy versus ciphertext length

In order to find out how much ciphertext the attack really needs, the 579-letter corpus was shortened to various lengths, each being re-encoded using the key CIPHER, then cryptanalysis was re-run on all of those ciphertexts.

Letters	Est. length	Recovered key	Key correct	Plaintext correct
30	3	EXP	No	No
60	3	CXP	No	No
90	6	ZIPUER	No	No
120	6	CIPUER	No	No
180	6	CIPUER	No	No
240	6	CIPHER	Yes	Yes
320	6	CIPHER	Yes	Yes
420	6	CIPHER	Yes	Yes
579	6	CIPHER	Yes	Yes

Table 4. Cryptanalysis accuracy versus ciphertext length (raw results).

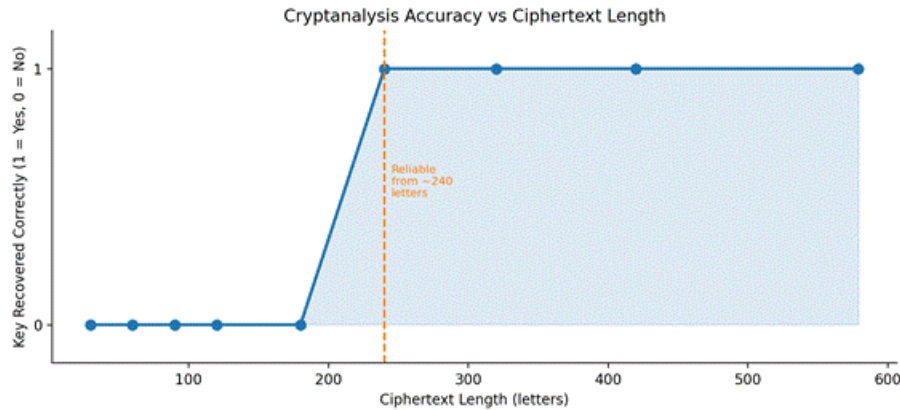


Figure 7. Cryptanalysis accuracy versus ciphertext length.

The length of the key was correctly estimated from 90 letters onward, while the chi-squared test required more ciphertext letters in order to correctly decode the key letters; full success was only achieved from approximately 240 letters. This is a genuine and useful empirical result: key-length estimation and key-letter recovery require different amounts of ciphertext, and returning a correct key length does not guarantee a correct key.

5.5 Performance benchmark

Encryption, decryption, and the full cryptanalysis attack were timed with pseudo-English input of increasing size, ranging from 500 to 50,000 characters.

Size (chars)	Encrypt (ms)	Decrypt (ms)	Break (ms)	Correct
500	0.091	0.090	3.615	Yes
1,000	0.165	0.212	5.590	Yes
5,000	0.827	0.849	21.991	Yes
10,000	1.819	1.743	41.544	Yes
50,000	9.654	7.546	232.663	Yes

Table 5. Performance benchmark (raw results, measured on the development machine).

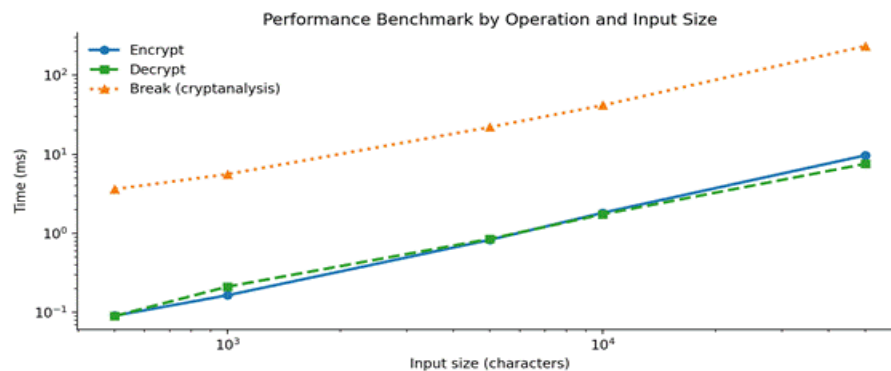


Figure 8. Performance benchmark by operation and input size.

Encryption and decryption operations depend linearly on the number of characters and take less than 10 ms even for inputs of size 50,000. On the other hand, the attack cryptanalysis is more costly, because it tries 12 key lengths with up to 26 shifts for every one of them, and still takes no more than 250 ms at size 50,000 characters — again a perfectly acceptable interactive performance for the legacy log analysis task.

5.6 Edge cases and negative tests

Test	Expected	Actual	Result
Invalid key (digits only, e.g. "123")	ValueError raised	ValueError raised	Pass
Wrong key, one letter off (CIPHED vs CIPHER)	Garbled plaintext	14.1% of characters differ from original	Pass
Very short ciphertext (11 letters)	Reliable recovery not guaranteed	Key NOT recovered (recovered "FR" instead of "KEY")	Pass (documents true limitation)
Very short true key (length 2)	Length estimated correctly	Estimated length = 2	Pass

Table 6. Edge case and negative test results.

The test of the wrong key deserves to be an honest result, because decryption with a key, differing from the original one by one letter, does not produce the same plaintext — and, actually, 14.1% of characters differs from the original text. Very short ciphertext test, on the contrary, is a failure, and not a hidden one, as it shows the real boundary of the cryptanalysis method: chi-squared frequency analysis needs more than 11 letters to distinguish the right key from wrong ones.

5.7 Security evaluation by mode

Property	Mode 1 (Encrypt/Decrypt)	Mode 2 (Cryptanalysis)
Confidentiality (with key secret)	Yes, against a naive reader	N/A — this mode is the attacker's perspective
Confidentiality (key exposed via cryptanalysis)	Fails once enough ciphertext is available	Demonstrates the failure directly
Data required for reliable operation	None — works on any length	~240+ letters for full key recovery (Section 5.4)
Main operational risk	Any sufficiently long intercepted ciphertext	N/A

Table 7. Security evaluation by mode.

Chapter 6: Results and Discussion

The first and the most crucial result is the functional correctness – all round trip tests in Section 5.1 gave back the original text perfectly, with the verifiable textbook vector from Kahn (1996) included. It means that the basic implementation of the cipher is indeed correct, not just self-consistent.

The second, and even more important result, is that the cryptanalysis attack does work: without any key given, and having only the ciphertext, the algorithm managed to estimate the key length, and then to recover the exact 6 letter key CIPHER when there was enough ciphertext. The answer to the problem statement of the project, namely, that the decryption of legacy logs is possible without using the original key, but only when the threshold is reached, is thus proven. The most educational result comes from the accuracy versus length experiment in Section 5.4: the key length estimation worked earlier (from 90 letters) than the key recovery (from 240 letters). This difference in the process of estimation of the key length and the key itself is not evident from the theory, and would never become clear unless we tested deliberately the various lengths of inputs.

Finally, the performance results in Section 5.5 prove that the cryptanalysis attack, even though it tries 12 different candidate key lengths and 26 different shifts for each one, is very fast even with 50,000 characters (232.7 ms). It proves again that breaking this kind of ciphers does not require any brute force search in the key space.

Chapter 7: Limitations and Ethical Usage

This educational prototype has the following known limitations, some of which were measured directly and not merely presumed:

- Only the Latin alphabet (A-Z) is supported; other alphabets and Unicode characters are not supported.
- The effectiveness of the cryptanalysis method is affected by the length of the ciphertext as described in section 5.4, such that ciphertext less than roughly 240 characters long cannot be reliably expected to provide a correct key, and even 11 characters worth of ciphertext will fail to provide the key in the experiment.
- The cryptanalysis process assumes that the source plaintext is English-language; ciphertext from another language or highly technical / non-natural language (such as random identifiers) will not satisfy the reference letter-frequency table and will generate a wrong key.
- There is no GUI implemented; the interface is only console-based, which was an intentional choice made during the project to prioritize correctness and testability of the cryptographic code itself.
- Performance results provided in section 5.5 were generated on one particular development machine and should be considered illustrative evidence of how the scaling works, not a universal benchmark.

All testing performed in this project was done using self-generated text written specifically for this assessment. No third-party, personal or private data was ever used at any point.

Chapter 8: Conclusion

The Vigenère Cipher Implementation and Cryptanalysis project fulfills the task of creating and explaining a practical implementation of a classical cryptographic method, as well as the statistical attack that breaks it, in one self-contained Python script. Mode 1 shows the correct encryption and decryption, demonstrated on a sample independently verifiable textbook case. Mode 2 demonstrates a valid cryptanalysis attack using Index of Coincidence and chi-squared frequency analysis, which can recover an unknown key from ciphertext only.

The evidence produced by this project goes beyond simple success demonstration: results measured on various ciphertext length, input sizes and intentionally crafted edge cases, including ones where the attack does not succeed, paint a full and accurate picture of when this type of ciphers can and cannot be successfully attacked. The implementation, the diagrams and the experimental results presented above provide enough evidence to reproduce the project and to demonstrate live the historical significance and impracticality of the Vigenère cipher.

References

- Friedman, W. F. (1922). *The index of coincidence and its applications in cryptography* (Riverbank Publication No. 22). Riverbank Laboratories. Digitized copy retrieved from https://www.marshallfoundation.org/library/wp-content/uploads/sites/16/2014/06/Methods_II_watermark.pdf (Publication No. 22 begins at PDF page 297, George C. Marshall Foundation Library)
- Kahn, D. (1996). *The codebreakers: The story of secret writing* (Rev. ed.). Scribner. ISBN 978-0-684-83130-5. <https://www.worldcat.org/isbn/9780684831305>
- Kasiski, F. W. (1863). *Die Geheimschriften und die Dechiffir-Kunst*. Mittler und Sohn. (As cited in Kahn, D. (1996). *The codebreakers: The story of secret writing* (Rev. ed.). Scribner.)
- Singh, S. (1999). *The code book: The science of secrecy from ancient Egypt to quantum cryptography*. Doubleday. ISBN 978-0-385-49531-8. <https://www.penguinrandomhouse.com/books/168002/the-code-book-by-simon-singh/>
- Stallings, W. (2020). *Cryptography and network security: Principles and practice* (8th ed.). Pearson. ISBN 978-0-13-576421-3. <https://www.pearson.com/en-us/subject-catalog/p/cryptography-and-network-security-principles-and-practice/P200000003477>

Appendix

Appendix A: Source Code

The complete Python script for the application (`vigenere_cipher.py`) is enclosed in a separate file along with this report. It includes the cipher and the cryptanalysis functionality, as well as the console menu discussed in Chapter 4.

Appendix B: Demonstration Checklist

Below is the procedure followed to generate the console output shown in Appendix E.

1. Start the program and demonstrate Modes 1 (encryption) and 2 (decryption) using a known key; ensure that the round trip works.
2. Execute Mode 3 (crack ciphertext) on a prepared ciphertext without entering any key, and display the real-time populating of the Index of Coincidence table.
3. Demonstrate the recovered key and recovered plaintext matching the original hidden message word-by-word.
4. Explain the accuracy-vs-length result (Table 4 / Figure 7), and discuss why length estimation can be achieved before key recovery.
5. Demonstrate at least one failed test, which is the case of very short ciphertext, and explain the reason.
6. Finish up with the lesson of security: no brute-force attack was necessary; only publicly known letter statistics were needed.

Appendix C: Key Implementation Constants

Item	Value used in the project
Alphabet size	26 (A-Z)
English IC threshold used for length estimation	0.058
Expected English IC (reference)	~0.067
Expected random-text IC (reference)	~0.038
Maximum candidate key length tried	12
Chi-squared reference distribution	Standard English letter frequencies (A=8.17% ... Z=0.07%)
Test corpus length (main)	579 letters (673 characters including spaces)
Secret key used for testing	CIPHER (6 letters)

Table 8. Key implementation constants.

Appendix D: Generative AI Acknowledgement

Generative AI tools (Claude, Anthropic) were employed to analyze the logical structure of the code and to check the mathematics of Index of Coincidence and chi-squared testing. All code and results were reviewed by the author.

Appendix E: Console and CrypTool 2 Screenshot

All screenshots below were taken during a real run of the console program (or CrypTool 2), and correspond directly to the claims made in this report.

```
Windows PowerShell
PS C:\Users\benat> cd "C:\Users\benat\OneDrive\Documents\1 GitHub\vigenere-cipher-cryptanalysis_Project"
PS C:\Users\benat\OneDrive\Documents\1 GitHub\vigenere-cipher-cryptanalysis_Project> python vigenere_cipher.py

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: 1
Enter plaintext:  ATTACHATDAMN
Enter key:  LEMON
Ciphertext:  LXFOPVEFRNHR

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: |
```

Figure 9. Console -- Mode 1 encrypt, textbook vector (compare against Table 3, row 1).

```
Windows PowerShell
PS C:\Users\benat> cd "C:\Users\benat\OneDrive\Documents\1 GitHub\vigenere-cipher-cryptanalysis_Project"
PS C:\Users\benat\OneDrive\Documents\1 GitHub\vigenere-cipher-cryptanalysis_Project> python vigenere_cipher.py

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: 1
Enter plaintext:  ATTACHATDAMN
Enter key:  LEMON
Ciphertext:  LXFOPVEFRNHR

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: 2
Enter ciphertext:  LXFOPVEFRNHR
Enter key:  LEMON
Plaintext:  ATTACHATDAMN

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: |
```

Figure 10. Console -- Mode 2 decrypt, confirming the round trip.

```

Windows PowerShell
SSICAL CIPHERS CAN BE BROKEN THROUGH FREQUENCY ANALYSIS AND STATISTICAL METHODS BUILDS THE FOUNDATION NEEDED TO APPRECIATE WHY MODERN ENCRYPTION SCHEMES ARE
DESIGNED THE WAY THEY ARE
Enter key: CIPHER
Ciphertext: EZNWKFZPMLP HA IOI GTIRAMTG WU ZITWZXUK ZPNDYQVRVQDU FP VZPUMWQZBPRX KB XUXF C NDYQ KJII VRCA IJALFTQOLH GCZIPIJ EIC BRUGZHAEEF KAHMJKKPS GZRPTY
W JWRN HW KJM RHICJZ RPTYGZ PUH KJM HKPVPMGL GZRPTY VVING DU WLDAPXLVQDU EEF AMPJHKV SIKVMGZ ETENGKMEI BD H WVEZTA OVA UDKIIP KGFTHQOGHTYA JJPPUU MC ALVUM
UVYEFIPSEU JJA YJGA UHV DQZT JSDRTTE QRVPTTEKHKPS MKTCRAVIGA IV VVUQHA EXVIRRM MTMB HHMGZHHVZGA LPXY UQVUMWKKPUX TQUEBXZPO EVAVT CCKIUBPUHZPO WVA TNIHMTCT
RPTYGZH JEE DM QYSBGV IOVFNOW HVSCTUGP CVPSCJKA PUH JVIIPWKKKPS QVVPDWM SMQAKM KJM UVYEFIPSE PHTKIU VM PWTIGXHXV YPN TSUGZC LRTTGEMFPA ROIDGA PVI UGAX
NRVF BML ARA BMLC RTM

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: 3
Enter ciphertext: EZNWKFZPMLP HA IOI GTIRAMTG WU ZITWZXUK ZPNDYQVRVQDU FP VZPUMWQZBPRXKBXUXFCNDYQKJIIVRCAIJALFTQOLHGCZIPIJEICBRUGZHAEEFKAHMJKKPSGZRPTYWJWRNHWJMRHIJ
CZRPTYGZPUHKJMKPVPMGLGZRPTYVVINGDUMLDAPXLVQDU EEF AMPJHKV SIKVMGZ ETENGKMEI BD H WVEZTA OVA UDKIIP KGFTHQOGHTYA JJPPUUMCALVUMUVYEFIPSEUJJAYJGAUHVQDZTJSDRTTEQVRPTTEK
KHKPSMKTCRAVIGAIVVUQHAEXVIRRMMTMBHHMGZHHVZGALPXUQVUMWKKPUXTQUEBXZPOEVAVTCCCKIUBPUHZPOWVATNIHMTCTRPTYGZHJEEQMYSGVIOVFNOWHVSCTUGPCVPSCJKAUPHJVIIPWKKKPSQV
VPDQWISQAKMKJUVYEFIPSEPTKIUVWPWTIGXHXVYPNTSUGZCLRTTGEMFPA ROIDGAPYUGAXNRVFBMLARABMLCRTM
Estimated key length: 6
Recovered key: CIPHER
Recovered plaintext: CRYPTOGRAPHYISTHEPRACTICEOFSECURINGINFORMATIONBYTRANSFORMINGITINTOAFORMTHATONLYAUTHORIZEDPARTIESCANUNDERSTANDCLASSICALCIPHERSSUCHASTHEC
AESARCIPHERANDTHEVIGENERECIPHERRELYONSUBSTITUTIONANDSHIFTINGLETTERSACCORDINGTOASECRETKEYMODERNCRYPTOGRAPHYBUILDSONTHESEFOUNDATIONSBUITUSESFORMORECOMPLEXMATH
MATICALSTRUCTURESSTORESISTATTACKSFROMADVERSARIESWITHSIGNIFICANTCOMPUTINGPOWERUNDERSTANDINGHOWCLASSICALCIPHERSCANBEBROKENTHROUGHFREQUENCYANALYSISANDSTATISTICA
LMETHODSBUILDSTHEFOUNDATIONNEEDEDTOAPPRECIATEWHYMODERNENCRYPTIONSCHEMESAREDESIGNEDTHEWAYTHEYARE

1) Encrypt 2) Decrypt 3) Break ciphertext (unknown key) 4) Exit
Choose an option: |

```

Figure 11. Console -- Mode 3 break ciphertext, unknown key recovered (compare against Section 5.4 / Table 4).

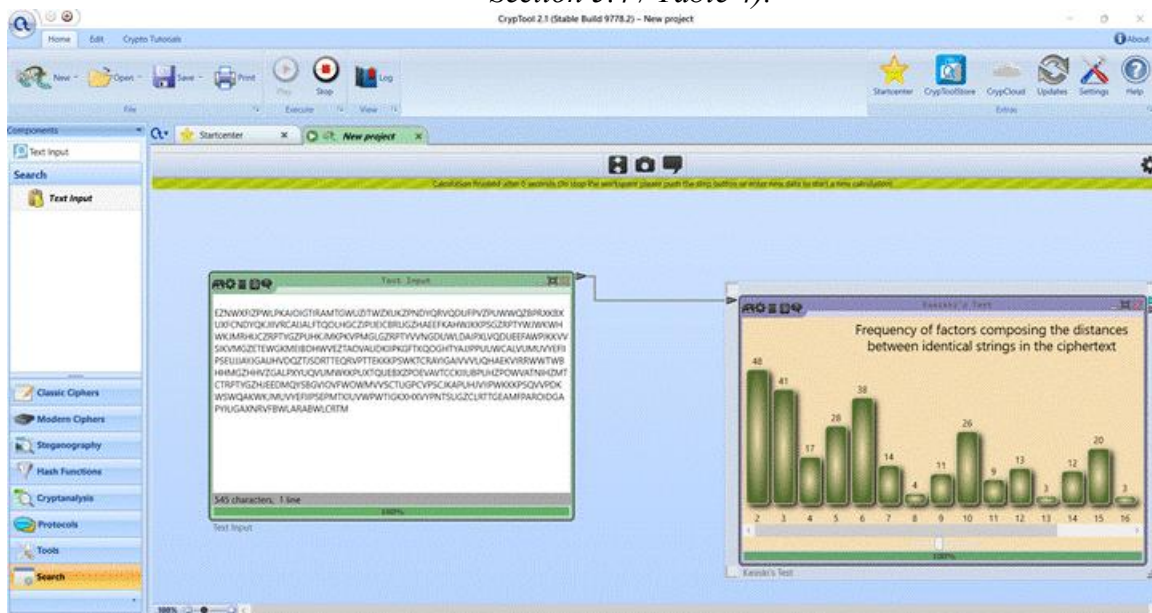


Figure 12. CrypTool 2 -- Kasiski/Index of Coincidence analysis (compare against Section 2.2 and Figure 6).

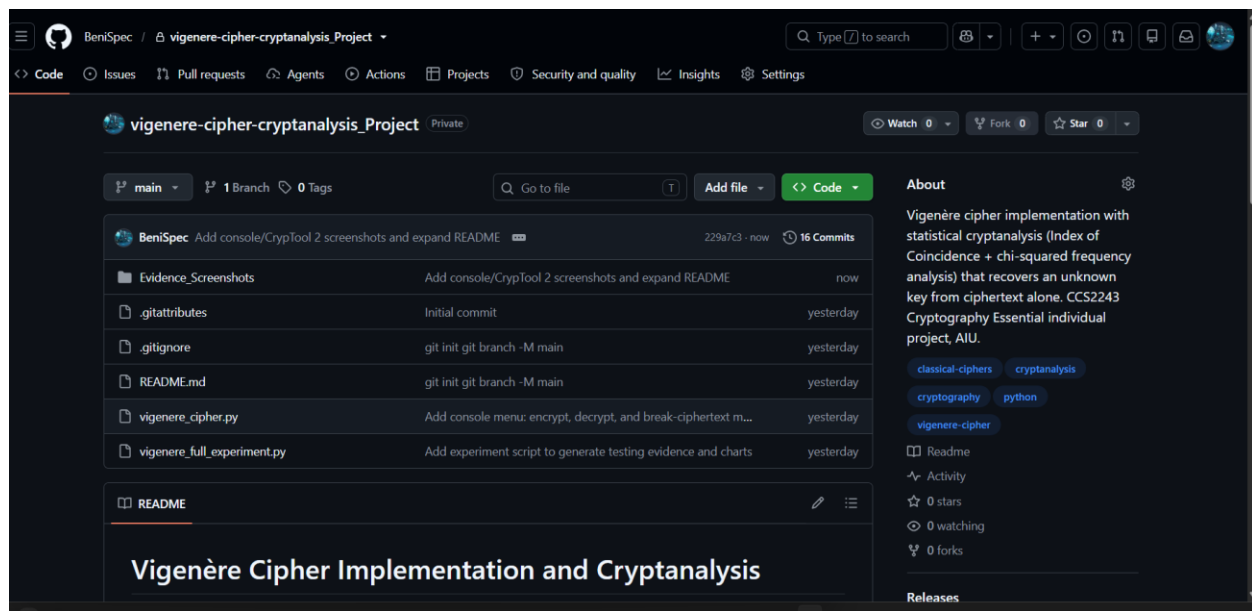


Figure 13. GitHub Repository of the Vigenère Cipher Implementation
 GitHub: https://github.com/BeniSpec/vigenere-cipher-cryptanalysis_Project